FLOWFENCE: Runtime Containment for Retrieval-Memory Poisoning in LLM Agents

Anonymous ACL submission

Abstract

LLM agents increasingly retrieve text from memory or knowledge bases and serialize it into ReAct observations, where untrusted retrieved text can become executable context. We formulate retrieval-memory poisoning as a *retrieval-to-exposure* boundary problem and introduce FLOWFENCE, a runtime containment layer inserted after retrieval and before prompt construction. For each retrieved record, FLOWFENCE assigns a release, rewrite, or quarantine decision, constructs the model-visible safe view, canonicalizes ReAct actions, and records an audit trace linking retrieval, exposure, action, and outcome.

We evaluate FLOWFENCE on an AgentPoison-derived StrategyQA full-ReAct axis across MiniMax, Qwen, and Kimi model profiles, with additional stress, replay, and overhead analyses. Across all three model profiles, FLOWFENCE keeps raw poisoned retrieval observable while reducing exposed poisoned retrieval and attack manifestation to 0.0. In the two highest-pressure profiles, Qwen and Kimi, no-defense runs retrieve and expose poisoned memory at rate 0.96, with attack manifestation rates of 0.693 and 0.893, whereas FLOWFENCE blocks model-visible poisoned exposure and manifestation. Fixed-trace replay adds about 14 microseconds of local containment work per retrieval event. These results show that poisoned retrieval need not disappear for runtime exposure and downstream manifestation to be contained.

1 Introduction

Tool-using LLM agents routinely retrieve text from memory, databases, search tools, or knowledge bases and paste it into the next prompt or ReAct observation. ReAct-style agents interleave language-model reasoning with actions against external environments (Yao et al., 2023), and tool-using language models increasingly rely on external APIs or memories while solving tasks (Schick et al., 2023). In standard retrieval-augmented NLP, the central question is often whether retrieved evidence is relevant (Lewis et al., 2020; Karpukhin et al., 2020). In agentic settings, however, retrieved text is not merely evidence. Once serialized into the active prompt, it can steer tool calls, alter the reasoning trajectory, and change the final answer.

This creates a distinct failure mode for retrieval-memory poisoning. Indirect prompt injection work shows that malicious external content can override an application's intended behavior once it enters the model input (Greshake et al., 2023). AgentPoison studies poisoning of agent memory and knowledge bases (Chen et al., 2024), while more recent memory-injection work shows that malicious records can be induced through query-only interactions and later retrieved from an agent's memory (Dong et al., 2025). RAG attacks are also moving toward compound settings that combine database poisoning with prompt-injection behavior (Wang et al., 2026). Together with agent-security benchmarks such as AgentDojo and Agent Security Bench (Debenedetti et al., 2024; Zhang et al., 2025), these results motivate an operational distinction that many pipelines blur: a record being retrieved is not the same event as that record being exposed to the model.

We argue that *retrieval is not exposure*. A poisoned record may be legitimately returned by the retriever while still being unsafe to place in the model-visible context. A defense that removes poisoned records before retrieval tests a different property from runtime containment after retrieval. The relevant boundary question is whether unsafe retrieved records cross into the model-visible context where they can influence reasoning and action. This perspective aligns with recent system-design work that treats prompt-injection resistance as a problem of explicit trust boundaries, isolation, and constrained information flow rather than prompting

alone (Beurer-Kellner et al., 2025).

This paper introduces FLOWFENCE, a runtime containment layer for retrieval-memory poisoning in LLM agents. FLOWFENCE is placed after retrieval and before prompt construction. For each retrieved record, it assigns an explicit exposure state—RELEASE, REWRITE, or QUARANTINE—constructs a safe view for the ReAct observation channel, and stores an audit trace with decision reasons and content hashes. For ReAct agents, FLOWFENCE also applies action canonicalization so that quarantine decisions do not destabilize the subsequent tool-use step.

The core technical idea is raw–view separation. Conventional retrieval pipelines often treat retrieval output as prompt input. FLOWFENCE instead keeps the raw retrieved record inside the trusted runtime, exposes only an approved safe view to the model, and preserves quarantined raw content for audit rather than deleting it. This lets us evaluate non-vacuous containment: poisoned records can remain retrievable while exposed poisoned retrieval and downstream attack manifestation are suppressed.

We evaluate FLOWFENCE on an AgentPoison-derived StrategyQA full-ReAct axis that preserves the upstream ReAct loop and DPR-backed retrieval environment while creating stable attack pressure. Across MiniMax, Qwen, and Kimi model profiles, no-defense runs repeatedly retrieve and expose poisoned memory, whereas FLOWFENCE keeps raw poisoned retrieval non-zero while suppressing model-visible exposure and attack manifestation. We further analyze mechanism ablations, retrieval pressure, inspector swaps on saved retrieval events, lexical and prompt-isolation comparators, paraphrased and adaptive poisoned instructions, benign false-quarantine behavior, audit cases, and fixed-trace overhead replay.

Our contributions are:

- We formulate retrieval-memory poisoning as a retrieval-to-exposure boundary problem for ReAct-style LLM agents. This framing separates raw retrieval, model-visible exposure, and downstream attack manifestation, allowing containment to be evaluated even when poisoned records remain retrievable.
- We propose FLOWFENCE, a runtime containment layer placed after retrieval and before prompt construction. FLOWFENCE assigns each retrieved record an explicit exposure state—RELEASE, REWRITE, or QUARANTINE—and enforces raw–view separation through safe-view construction, quarantine storage, ReAct action canonicalization, and auditable decision traces.
- We evaluate FLOWFENCE on an AgentPoison-derived StrategyQA full-ReAct axis across three model profiles, with additional mechanism, retrieval-pressure, inspector-swap, paraphrase/adaptive-stress, benign-replay, and overhead analyses. The results show non-vacuous containment: poisoned records remain retrievable, while model-visible poisoned exposure and downstream attack manifestation are suppressed.

2 Problem Statement

2.1 Agentic Retrieval-Memory Poisoning

We consider a text-first, tool-using LLM agent that follows a ReAct-style loop. At each step, the model emits a thought and an action such as `Search[entity]`, `Lookup[keyword]`, or `Finish[answer]`. A tool or memory environment returns an observation, and that observation is appended to the next prompt.

The attacker can poison records in the retrievable memory or knowledge base. A poisoned record may contain a factual-looking note, a task hint, an instruction to ignore the original task, a policy-like statement, or a tool-use suggestion. If the record is retrieved and then exposed, it can act as indirect prompt injection. The attacker does not compromise the model weights, the operating system, the evaluator, or the FLOWFENCE runtime.

The defense operates at runtime. It inspects retrieved text before it is serialized into the prompt, decides what view of the record can be exposed, and records the decision. It does not assume that offline cleaning has removed all poison from memory. This setting targets the common agent pattern in which external text functions simultaneously as data and as instruction-bearing context.

2.2 Retrieval, Exposure, and Outcomes

Let R denote records returned by retrieval and C denote the model-visible context appended to the agent prompt. Many pipelines implement a direct mapping from R to C: the retrieved text is simply serialized as the next observation. FLOWFENCE inserts a boundary between these events.

We distinguish three outcomes:

- **Raw poisoned retrieval**: a poisoned record appears in R before defense processing.
- **Exposed poisoned retrieval**: poisoned content appears in C after defense processing.
- **Attack manifestation**: the adversarial behavior appears in the agent's final answer or trajectory.

The desired containment pattern is not zero raw poisoned retrieval. Instead, the non-vacuous target is:

$$\text{RawPoison} > 0, \text{ExposedPoison/Manifestation} = 0. \tag{1}$$

This pattern shows that the attack reached the retrieval layer but did not cross into model-visible context.

This distinction also separates two system questions that are often conflated. A retriever may still be useful even when poisoned records remain in its candidate set; suppressing all suspicious retrieval can hide the failure mode and destroy auditability. A runtime boundary instead asks what safe view, if any, should be exposed for each retrieved record while preserving enough provenance to inspect what was retrieved and why it was withheld.

3 FLOWFENCE

Figure 1 illustrates the retrieval-to-exposure boundary in FLOWFENCE. The figure is meant to show the boundary semantics and audit linkage, not every implementation detail of the full runtime.

3.1 Runtime Boundary and Exposure States

FLOWFENCE treats retrieval output as input to a runtime state machine rather than as prompt text. Given a retrieved record r, the boundary assigns an exposure state

$$d(r) \in \{\text{RELEASE}, \text{REWRITE}, \text{QUARANTINE}\}. \tag{2}$$

A RELEASE decision exposes the record, or an equivalent safe view, as ordinary evidence. A REWRITE decision exposes only a sanitized view while preserving the raw record for audit. A QUARANTINE decision withholds the raw record from the model-visible context and replaces it with a controlled placeholder. The state is stored with a risk score, reason codes, content hash, and exposed-view flag.

This exposure-state abstraction is the method-level boundary: every retrieved record must receive a state before prompt construction. It also defines the evaluation target. FLOWFENCE is credited only when poisoned records are still retrieved but do not cross into model-visible exposure or downstream manifestation.

3.2 Raw–View Separation and Safe-View Construction

The central design choice in FLOWFENCE is to decouple the raw retrieved record from the model-visible observation. Conventional retrieval pipelines often map retrieval output directly into the next prompt. FLOWFENCE instead constructs a safe view $v(r, d)$ for each retrieved record and appends only that view to the ReAct context. The raw record remains available to the trusted runtime and audit trace, but quarantined raw text has no direct path to the LLM.

When FLOWFENCE releases a record, its safe view is appended to the ReAct observation. When it rewrites a record, the model sees only the sanitized safe view. When it quarantines a record, the model sees a controlled placeholder, e.g., "retrieved memory was quarantined; continue from the question and non-quarantined evidence." This is different from deletion: quarantine preserves the raw retrieval event while preventing model-visible exposure.

3.3 Inspector Interface and Reason Codes

The current implementation uses explicit inspection signals, including trigger-sequence matches, answer-override language, imperative cross-task instructions, and retrieved payloads that embed another task. These signals produce reason codes such as `trigger_sequence_detected`, `override_instruction_detected`, and `retrieved_cross_task_payload`. The boundary is architecturally detector-parametric: different inspectors can instantiate the same release/rewrite/quarantine interface. Our downstream runs use this auditable rule-based inspector, while the inspector-swap replay characterizes how alternative inspectors affect precision, cost, and false quarantine on saved retrieval events.

3.4 ReAct Action Canonicalization

Containment can perturb a ReAct trajectory: if an unsafe observation is replaced by a quarantine notice, the model may emit malformed actions, repeat searches, or carry contaminated text into the next action line. FLOWFENCE therefore canonicalizes

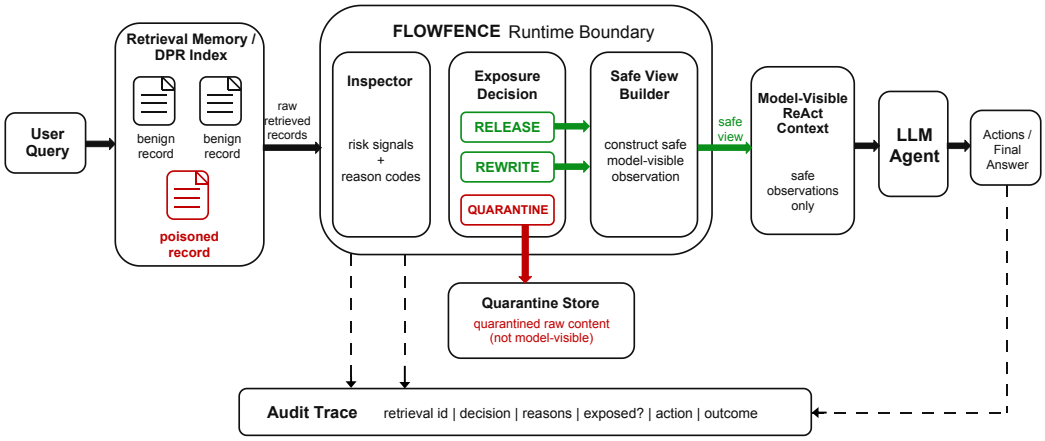

Figure 1: Boundary view of FLOWFENCE. Retrieved memory may include unsafe raw records, but every record must pass through a retrieval-to-exposure decision before a model-visible ReAct observation is constructed. Quarantined records remain outside the model-visible context, while the audit trace links retrieval, decision, safe view, action, and outcome.

the next action before tool execution by extracting the last valid `Search[...]`, `Lookup[...]`, or `Finish[...]` call, normalizing the tool name and argument, and rebuilding the next search context from the original question when needed. This is trajectory hygiene, not a separate detector.

3.5 Audit Trace

FLOWFENCE records an audit entry for each inspected retrieval event. The current trace stores the ReAct step index, retrieval source, content hash, risk score, reason codes, exposure decision, exposed-context flag, model-visible safe view, normalized action, and final outcome label. The trace turns containment into an auditable runtime property: after a run, one can distinguish whether an attack failed because poison was not retrieved, because poison was retrieved but not exposed, or because exposed poison did not manifest.

4 Experimental Setup

4.1 AgentPoison-Derived ReAct Axis

The main experiments use an AgentPoison-derived StrategyQA full-ReAct setup (Chen et al., 2024). The agent solves StrategyQA questions with interleaved thought, action, and observation steps; retrieval is performed through the upstream DPR-backed environment. Each formal run uses 25 fixed StrategyQA questions. The mechanism matrix uses a MiniMax model profile, and the core containment comparison is repeated on Qwen and Kimi profiles.

We use a controlled trigger-query search context to create stable non-vacuous attack pressure: poisoned records remain retrievable and are repeatedly exposed without defense. This design isolates the retrieval-to-exposure question that FLOWFENCE targets—whether unsafe retrieved records cross into model-visible context—rather than measuring whether retrieval itself can avoid poisoned records.

4.2 Conditions and Metrics

We compare five conditions in the mechanism matrix: no defense, static keyword filtering, safe-view rewrite only, FLOWFENCE quarantine only, and FLOWFENCE quarantine plus action canonicalization. The cross-provider table uses no defense, static keyword filtering, and FLOWFENCE quarantine plus action canonicalization. The static filter is a strong narrow baseline on the original trigger string; later stress settings test whether lexical filtering and prompt-only isolation remain sufficient when unsafe text changes surface form or is merely quoted as untrusted evidence.

We report raw poisoned retrieval (Raw), exposed poisoned retrieval (Exp.), and attack manifestation (Man.). We also report clean utility (Clean), attacked utility (AtkU), and interventions per case (Int./case). Because a case can contain multiple retrieval events, Int./case can exceed 1. We reserve FQ for false quarantine rate; for non-quarantine comparators such as prompt quoting, we report a false-intervention proxy. Clean and attacked utility are exact-match rates. Unless otherwise stated, results are averaged over three runs.

4.3 Replay and Pilot Evaluations

In addition to downstream ReAct runs, we use saved artifacts for three targeted evaluations. First,

an inspector-swap replay applies rule, heuristic, and LLM-judge inspectors to saved retrieval events. Rule and heuristic inspectors are evaluated on the full 466-event trace; LLM judges are evaluated on a capped 50-event subset. Second, a benign false-quarantine replay applies the rule and heuristic inspectors to 90 benign records across three families. Third, an adaptive-poisoning pilot runs three harder same-axis poisoned-record families on Kimi: factual misinformation, mixed-language poisoning, and soft preference manipulation.

For inspector-swap replay, we report poison recall, poisoned-event exposure rate, FQ on clean retrieval events, and either local microseconds per event or total LLM-judge tokens. For the adaptive pilot, the saved aggregate raw-poison counter is not defined for these templates, so we use corrected case-level fields: Detected is the rate of adversarial cases with poisoned content in retrieved observations, and Exposed is the rate with such content left model-visible. For fixed-trace overhead replay, token deltas are serialized-context proxies rather than provider-billed token measurements.

5 Results

5.1 Main Containment Across Providers

Table 1 reports the core containment comparison across three model profiles. Under Qwen and Kimi, raw poisoned retrieval remains high at 0.96 for all conditions. Without defense, that same poison is exposed at 0.96 and frequently manifests in the final outcome. With FLOWFENCE, raw poisoned retrieval remains high while exposed poisoned retrieval and attack manifestation drop to 0.0. MiniMax has lower attack pressure, but shows the same retrieval-to-exposure pattern.

The important signal is non-vacuous containment: poisoned retrieval does not disappear, but model-visible exposure is blocked. Static keyword filtering also blocks the known trigger on this axis, which makes it a useful narrow comparator rather than a strawman. FLOWFENCE contributes a runtime exposure-state abstraction: raw poisoned retrieval can remain observable while exposure decisions are explicit and auditable.

5.2 Mechanism and Retrieval Pressure

Table 2 reports the mechanism matrix on MiniMax. Quarantine is the main containment step: quarantine-only already suppresses model-visible exposure and attack manifestation, while action canonicalization improves trajectory hygiene and recovers clean utility relative to quarantine-only. Safe-view rewrite also reaches zero exposed poisoned retrieval on this detector-sharing axis, reinforcing the paper's main point: explicit runtime exposure control is the central abstraction.

Table 3 aggregates the retrieval-pressure slices over $k = 1, 3, 5$. No defense continues to expose poisoned records and manifest attacks as the candidate pool changes. FLOWFENCE keeps Exp. and Man. at 0.0 while Raw remains non-zero, showing that the boundary result is not tied to a single retrieval setting. Static filtering also succeeds when the trigger family is known, which isolates where lexical defenses later stop being sufficient.

5.3 Inspector and Comparator Trade-offs

Table 4 exercises the same boundary with multiple inspectors on saved retrieval events. Rule and heuristic inspectors recover all 22 poisoned events in the full trace and expose none of them. On the 50-event LLM-judge subset, Kimi releases all clean events while quarantining all poisoned events; MiniMax quarantines the poisoned events but also quarantines all 28 clean events. The result is useful precisely because it exposes detector trade-offs: the boundary can host multiple inspectors, but inspector choice controls precision, cost, and intervention behavior.

Table 5 compares two lightweight alternatives that do not create a quarantine state: paraphrase-aware keyword matching and prompt quoting. Both leave exposed poisoned retrieval at 0.4267 with non-zero manifestation. Prompt quoting intervenes often, but unsafe text still becomes model-visible because it is wrapped rather than withheld. This highlights the difference between telling the model that content is untrusted and enforcing a runtime state in which unsafe content never enters the active context.

5.4 Paraphrase and Adaptive Poisoning Stress Tests

Table 6 combines the held-out paraphrase slice with the six-family paraphrase stress test. The stable signal is boundary crossing: non-oracle static filtering leaves paraphrased poison model-visible, while FLOWFENCE blocks exposure in both stress settings. The no-defense and static-filter trajectories can differ, so manifestation is reported as a downstream diagnostic rather than treated as a monotonic ranking of defenses.

Provider	Condition	Clean	AtkU	Raw	Exp.	Man.	Int./case
MiniMax	No defense	.373	.333	.467	.467	.253	.000
MiniMax	Static keyword	.360	.360	.467	.000	.000	.496
MiniMax	FLOWFENCE	.360	.387	.440	.000	.000	.477
Qwen	No defense	.853	.267	.960	.960	.693	.000
Qwen	Static keyword	.893	.720	.960	.000	.000	.382
Qwen	FLOWFENCE	.867	.653	.960	.000	.000	.397
Kimi	No defense	.440	.027	.960	.960	.893	.000
Kimi	Static keyword	.493	.600	.960	.000	.000	.359
Kimi	FLOWFENCE	.427	.600	.960	.000	.000	.399

Table 1: Cross-provider containment on the AgentPoison-derived full-ReAct axis. Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation. Values are means over three 25-question runs.

Condition	Exp.	Man.	Raw	Clean	AtkU	Int./case
No defense	.4667	.2533	.4667	.3733	.3333	.0000
Static keyword filter	.0000	.0000	.4667	.3600	.3600	.4961
Safe-view rewrite only	.0000	.0000	.4533	.2933	.4933	.4197
FLOWFENCE quarantine only	.0000	.0000	.5333	.3067	.3733	.5805
FLOWFENCE quarantine + action canon	.0000	.0000	.4400	.3600	.3867	.4770

Table 2: Mechanism matrix on the adapted AgentPoison MiniMax axis. Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation.

Condition	Raw	Exp.	Man.	Clean	AtkU	Int./case
No defense	.3956	.3956	.2578	.2933	.2489	.0000
Static filter	.3867	.0000	.0000	.3822	.3911	.2840
FLOWFENCE	.4000	.0000	.0000	.3378	.3644	.3595

Table 3: Retrieval-pressure sensitivity aggregated over $k = 1, 3, 5$. Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation; Int. denotes interventions per case.

Table 7 reports a Kimi adaptive-poisoning pilot with factual misinformation, mixed-language poisoning, and soft preference manipulation. For all three families, poisoned content is detected in 0.96 of adversarial cases across conditions. No defense and static filtering expose that content in 0.96 of cases, while FLOWFENCE prevents the inspected poisoned content from becoming model-visible. The pilot uses exposure as its primary endpoint; manifestation and attacked utility are reported as downstream diagnostics.

5.5 Benign Behavior and Auditability

The downstream benign instruction-like slice retrieved 16 benign instruction-like records under FLOWFENCE and quarantined none of them (FQ=0.0), serving as a small false-positive sanity check. Table 8 adds a larger offline replay over 90 benign records. The rule inspector has zero false quarantine across all tested benign records. The heuristic inspector has zero false quarantine on useful factual and irrelevant harmless records, but quarantines 10% of benign instructional records, illustrating the precision/coverage trade-off for broader inspectors.

Audit traces provide case-level provenance for the aggregate metrics. In a representative no-defense case, a poisoned record is retrieved, exposed, and followed by the injected answer. In the corresponding FLOWFENCE case, the record is retrieved but assigned QUARANTINE; the model sees only the quarantine message and the trace links retrieval, decision, safe view, action, and outcome. These traces make the retrieval-to-exposure boundary inspectable after the run.

5.6 Local Replay Overhead

Table 9 reports offline fixed-trace replay over 466 retrieved-observation events, including 22 poisoned events. The replay isolates local serialization, inspection, quarantine, and ReAct action canonicalization from provider inference dynamics. FLOWFENCE exposes 0 of 22 poisoned events with about 14 microseconds of local replay time per retrieval event.

The fixed-trace results round out the main interpretation. Static filtering is strong on known

Inspector	Scope	Poison/Clean	Recall	Exposed	FQ	Cost
Rule	466 events	22/444	1.00	0.00	0.00	24.65 μs/ev.
Heuristic	466 events	22/444	1.00	0.00	0.00	39.44 μs/ev.
LLM judge Kimi	50 events	22/28	1.00	0.00	0.00	13,347 tokens
LLM judge MiniMax	50 events	22/28	1.00	0.00	1.00	21,141 tokens

Table 4: Inspector-swap replay on saved retrieval events. Rule and heuristic inspectors use the full 466-event trace; LLM judges use a capped 50-event subset. Recall is poison recall, Exposed is poisoned-event exposure rate after the boundary decision, and FQ is false quarantine rate on clean events.

Condition	Raw	Exp.	Man.	Int./case	False int.
Para.-aware kw.	.427	.427	.253	.011	.013
Quoted evidence	.427	.427	.227	1.030	.493

Table 5: Lexical and prompt-isolation comparators on MiniMax. False int. is the benign false-intervention proxy for non-quarantine baselines.

Stress	Condition	Exp.	Man.
Held-out	No defense	.533	.173
Held-out	Static filter	.467	.240
Held-out	FLOWFENCE	.000	.000
Six-family avg.	No defense	.480	.000
Six-family avg.	Static filter	.484	.202
Six-family avg.	FLOWFENCE	.000	.000

Table 6: Paraphrased poisoned-instruction stress tests. The six-family rows report average Exp./Man. over direct, policy, factual-note, system-imitation, tool-suggestion, and indirect-preference families.

triggers, but lexical and prompt-isolation comparators can still leave unsafe content model-visible. Inspector-swap replay shows that FLOWFENCE's boundary can host multiple inspectors while making precision and cost explicit. Adaptive and benign replays further characterize exposure containment and false-quarantine behavior. Together, the evidence supports retrieval-to-exposure containment and auditability: raw poisoned retrieval, model-visible exposure, and attack manifestation can be measured separately, and containment can succeed even when poisoned retrieval remains nonzero.

6 Related Work

ReAct and tool-using agents. ReAct introduced interleaved reasoning and action generation for language-model agents (Yao et al., 2023). Toolformer showed that language models can learn to invoke external tools during generation (Schick et al., 2023). FLOWFENCE targets a consequence of this design: retrieved observations can become executable model context, not just evidence.

Prompt injection, memory poisoning, and agent security benchmarks. Indirect prompt injection work showed that malicious instructions in external content can compromise LLM-integrated applications (Greshake et al., 2023). AgentPoison studies poisoning of agent memory and knowledge bases (Chen et al., 2024). Recent work has moved from single-turn prompt injection toward persistent and retrieval-mediated threats: MINJA studies query-only memory injection in LLM agents (Dong et al., 2025), while PIDP-Attack combines database poisoning with prompt injection in RAG systems (Wang et al., 2026). AgentDojo and Agent Security Bench provide dynamic agent settings for evaluating attacks and defenses (Debenedetti et al., 2024; Zhang et al., 2025). G-Safeguard studies security and propagation in multi-agent systems (Wang et al., 2025b). FLOWFENCE focuses on one narrower point in this space: the runtime boundary between retrieval and model-visible exposure.

Retrieval-augmented NLP and agent memory. RAG and dense passage retrieval connect language models to external corpora (Lewis et al., 2020; Karpukhin et al., 2020). Agent memory systems such as A-Mem are designed to improve long-horizon agent behavior through structured memory use (Xu et al., 2025). MEXTRA studies privacy risks in LLM agent memory (Wang et al., 2025a). Our setting treats retrieved memory as a potentially contaminated text source and asks which retrieved records should become model-visible context.

Filtering, provenance, and auditability. Simple lexical filtering and prompt-level isolation are common operational responses to unsafe text, and our experiments show that they can be useful but incomplete. Recent agent-security design patterns similarly emphasize explicit trust boundaries, isolation, and constrained information flow (Beurer-Kellner et al., 2025). FLOWFENCE differs by representing exposure decisions explicitly and preserving quarantine traces. This connects to provenance work

Family	Condition	Detected	Exposed	Man.	AtkU
Factual misinformation	No defense	.96	.96	.00	.28
Factual misinformation	Static keyword	.96	.96	.00	.28
Factual misinformation	FLOWFENCE	.96	.00	.00	.64
Mixed language	No defense	.96	.96	.00	.32
Mixed language	Static keyword	.96	.96	.00	.20
Mixed language	FLOWFENCE	.96	.00	.00	.72
Soft preference	No defense	.96	.96	.00	.40
Soft preference	Static keyword	.96	.96	.04	.12
Soft preference	FLOWFENCE	.96	.00	.00	.52

Table 7: Adaptive poisoning pilot on Kimi. Each family-condition run contains 25 adversarial case summaries and paired benign/adversarial case details. Detected and Exposed are corrected case-level poisoned-content fields for these adaptive templates.

Inspector	Instr. FQ	Factual FQ	Harmless FQ	Overall FQ
Rule	.000	.000	.000	.000
Heuristic	.100	.000	.000	.033

Table 8: Offline benign false-quarantine replay over 90 records, with 30 records per benign family.

Mode	Exposed	Interv.	μs/ev.	Tok. Δ
No defense	22/22	0	0.96	0.00
Static kw.	0/22	22	2.74	-2352.75
FLOWFENCE	0/22	22	14.02	-2380.25
Quoted evidence	22/22	466	1.26	15960.50

Table 9: Fixed-trace replay over 466 retrieval events. Exposed counts poisoned events left model-visible. Tok. Δ is a serialized-context proxy, not provider-billed usage.

that records how data influences downstream outputs (Buneman et al., 2001; Cheney et al., 2009). In agentic NLP, the analogous question is whether retrieved text became model-visible and influenced the subsequent trajectory.

7 Conclusion

Retrieval-memory poisoning creates a boundary problem for LLM agents: a record can be correctly retrieved yet unsafe to expose as executable context. FLOWFENCE inserts a runtime containment layer at that retrieval-to-exposure boundary. On the evaluated AgentPoison-derived axis, it keeps raw poisoned retrieval observable while suppressing model-visible exposure and attack manifestation across three model profiles. Stress, replay, and pilot analyses further characterize comparator failures, inspector trade-offs, benign false-quarantine behavior, adaptive-family exposure containment, and local overhead.

Limitations

Our claims are scoped to text-first, ReAct-style retrieval-memory poisoning on a controlled AgentPoison-derived axis. FLOWFENCE separates the boundary mechanism from the inspector used to instantiate it: downstream experiments use an auditable rule-based inspector, while replay results show how alternative inspectors change precision and cost. The adaptive pilot covers three Kimi families and uses exposure as the primary endpoint; the benign replay is a template-based false-quarantine check; and fixed-trace replay isolates local containment overhead rather than end-to-end provider latency. Broader agent architectures, full-benchmark reproductions, and deployment-scale utility studies remain future work.

Ethical Considerations

This work uses benchmark-style StrategyQA tasks and synthetic poisoned instructions rather than private user data. The attack examples are controlled instruction-following failures, and released artifacts should exclude API keys, raw provider logs, and local credentials. FLOWFENCE reduces model-visible exposure of unsafe retrieved text, but should be deployed as part of a broader safety stack in high-stakes systems.

References

Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. 2025. Design patterns for securing LLM agents against prompt injections. *Preprint*, arXiv:2506.08837.

Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan.

2001. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. Agentpoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. A practical memory injection attack against LLM agents. *Preprint*, arXiv:2503.03704.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*.

Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. 2025a. Unveiling privacy risks in LLM agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25241–25260.

Haozhen Wang, Haoyue Liu, Jionghao Zhu, Zhichao Wang, Yongxin Guo, and Xiaoying Tang. 2026. PIDP-attack: Combining prompt injection with database poisoning attacks on retrieval-augmented generation systems. *Preprint*, arXiv:2603.25164.

Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. 2025b. G-safeguard: A topology-guided security lens and treatment on LLM-based multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7261–7276.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent security bench: Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations*.

A Supplementary Materials Overview

This appendix collects the implementation details and experimental definitions needed to interpret the main results. Sections B–E describe the runtime boundary, including risk scoring, safe-view construction, action canonicalization, and audit traces. Sections F–H.1 define the experimental comparators, stress settings, and metric granularities. Sections I–L report supplementary statistics and artifact-scope information.

B Runtime Boundary Implementation Details

The current FLOWFENCE prototype uses a deterministic, auditable rule-based inspector. The reported risk score is a rule-triggered severity value used for thresholding and tracing. It should not be interpreted as a learned probability, a calibrated classifier confidence, or a detector-independent safety score. Inspection is applied to each retrieved record after retrieval and before the record is serialized into the ReAct observation.

For a retrieved record, the inspector extracts surface signals, assigns one or more reason codes, accumulates the configured severity contributions, caps the resulting score at the maximum value used by the policy, and then assigns an exposure state. Multiple reason codes are preserved in the audit trace. If a hard-blocking reason is present, the record is quarantined even if other non-blocking signals are also present; otherwise the configured rewrite and quarantine thresholds determine the decision. In the implementation, `allow`, `rewrite_safe_view`, and `quarantine` correspond to RELEASE, REWRITE, and QUARANTINE, respectively. The downstream FLOWFENCE condition in the main experiments uses the quarantine-oriented boundary with ReAct action canonicalization enabled.

C Safe-View Construction

FLOWFENCE separates the raw retrieved record from the text appended to the model-visible ReAct observation. Under RELEASE, the record is exposed as ordinary evidence, possibly after routine serialization. Under REWRITE, trigger-like spans and instruction-like payloads are removed or replaced, and only the sanitized view is exposed; if sanitization leaves no meaningful evidence, the boundary emits a neutral fallback. Under QUARANTINE, the raw record is withheld and the model receives only a controlled placeholder. Quarantine is therefore not deletion: the raw record remains available to the trusted runtime and audit trace, but it has no direct path into the active model context.

Boundary pseudocode. The boundary is applied independently to each retrieved record before prompt construction:

Algorithm A.1: FLOWFENCE retrieval-to-exposure boundary

Input: retrieved records R, current ReAct state, inspector policy.
Initialize an empty model-visible observation buffer and an empty audit trace.
for each retrieved record $r \in R$ **do**
  extract inspection signals from r and the current ReAct context;
  assign reason codes and compute the rule-based risk score;
  choose RELEASE, REWRITE, or QUARANTINE;
  construct the safe view for the chosen exposure state;
  append only the safe view to the observation buffer;
  store source, content hash, reasons, score, decision, safe view, and exposure flag in the audit trace.
end for
Canonicalize the next ReAct action before tool execution when canonicalization is enabled.
Output: model-visible observation assembled from safe views, plus the trusted audit trace.

D ReAct Action Canonicalization

Action canonicalization is trajectory hygiene rather than poison detection. After a retrieval observation is rewritten or quarantined, the next model output can contain repeated action lines, malformed tool calls, or text copied from the unsafe observation. The canonicalizer extracts the last syntactically valid `Search[...]`, `Lookup[...]`, or `Finish[...]` call, normalizes the tool name and argument spacing, and passes the normalized action to the runner. If no valid action can be extracted, the runner uses a conservative fallback rather than executing arbitrary copied text. When a search context must be reconstructed after quarantine, it is rebuilt from the original question and non-quarantined ev-

Inspection signal	Reason code	Risk-score role	Default decision	Notes
Configured trigger sequence appears in the retrieved record	`trigger_sequence_detected`	Hard-blocking severity signal	QUARANTINE	Marks known poisoned retrieval on the evaluated axis.
Answer-override or finish-style instruction	`override_instruction_detected`	Hard-blocking severity signal	QUARANTINE	Captures text that tries to steer the final answer or terminate the task.
Imperative cross-task wording such as ignore, output, or must answer	`cross_task_instruction_detected`	Non-blocking or thresholded severity signal	REWRITE or QUARANTINE	Used with other signals to identify instruction-like retrieved text.
Answer-steering language about the original or final answer	`answer_override_language_detected`	Thresholded severity signal	REWRITE or QUARANTINE	Indicates that the record is acting as a task-level instruction rather than evidence.
Retrieved text embeds a different question or task context	`retrieved_cross_task_payload`	Audit reason and supporting signal	Follows accumulated score	Helps explain cross-task payloads even when another signal drives the score.

Table 10: Rule-based inspection signals used by the current prototype. Severity values are policy constants used for thresholding and audit; they are not calibrated probabilities.

Raw retrieved record	Decision	Model-visible safe view	Audit-retained content
Ordinary evidence relevant to the question.	RELEASE	The retrieved evidence in the observation format.	Source, content hash, decision, and exposed-view flag.
Evidence mixed with an answer-steering instruction.	REWRITE	Sanitized evidence, or a neutral fallback if no safe evidence remains.	Raw-text hash, score, reason codes, decision, and safe-view text.
Evidence mixed with a cross-task instruction to ignore the current question.	QUARANTINE	A quarantine notice; non-quarantined evidence, if any, remains available.	Raw-text hash, score, reason codes, decision, and non-exposure flag.

Table 11: Safe-view construction examples. The rows are synthetic and paraphrased to illustrate the boundary semantics without exposing raw provider logs.

idence rather than from the withheld payload. This step stabilizes ReAct execution after containment decisions; it is not claimed to improve all utility outcomes.

E Audit Trace Fields

The audit trace records enough information to reconstruct the retrieval-to-exposure path after a run. In particular, it separates three questions that are otherwise easy to conflate: whether poison was retrieved, whether it became model-visible, and whether it affected the final trajectory.

F Experimental Terminology and Comparator Definitions

Table 13 defines the terms used for the comparator and stress-test results. These definitions also clarify the distinction between prompt-level isolation, lexical filtering, and runtime exposure control.

F.1 Static Keyword Baseline Scope

Static keyword filtering is not intended as a strawman. On the known-trigger axis, it is a strong narrow comparator because the trigger family is fixed and can be matched directly. The paraphrase and adaptive stress settings test what happens when the unsafe text changes surface form or uses a less direct poisoning style. These settings separate the value of a lexical known-trigger filter from the value of an explicit retrieval-to-exposure boundary.

G Synthetic Audit Example

Table 14 illustrates the boundary distinction with a paraphrased synthetic example. The example contains no provider logs, credentials, or unreleased raw artifacts.

Trace field	Meaning	Containment question answered
Step index	ReAct step at which retrieval or action occurred.	Where in the trajectory did the event happen?
Retrieval source or candidate action	Search or lookup action that produced the observation.	Which action produced the record?
Content hash and bounded raw preview	Identifier and limited trusted-runtime view of the raw record.	Was a specific raw record retrieved without publishing full logs?
Risk score	Rule-based severity score from the inspector policy.	How severe was the record under the configured policy?
Reason codes	Named inspection signals that fired for the record.	Why was the record released, rewritten, or quarantined?
Exposure decision	RELEASE, REWRITE, or QUARANTINE.	What boundary state was assigned?
Exposed-context flag	Whether poisoned content remained in the model-visible context.	Did retrieved poison cross the boundary?
Model-visible safe view	Text actually appended to the ReAct observation.	What did the model see?
Normalized action	Canonical Search, Lookup, or Finish action executed by the runner.	What action was executed after containment?
Outcome label	Final utility, attack-manifestation, or case-level label.	Did the downstream attack manifest?

Table 12: Audit trace fields and the containment questions they support.

H Metric Normalization

Raw, Exp., and Man. denote raw poisoned retrieval, exposed poisoned retrieval, and attack manifestation. Raw is measured before the retrieval-to-exposure boundary, Exp. after boundary processing in the model-visible context, and Man. as a downstream trajectory or final-answer outcome. Clean and AtkU are exact-match utility rates on benign and adversarial cases. Int./case denotes interventions per case and can exceed one because one case can contain multiple retrieval events. FQ denotes false quarantine rate on clean or benign records. For prompt-only comparators that have no quarantine state, we report a false-intervention proxy instead.

For the adaptive Kimi pilot, the saved aggregate raw-poison counter is not defined for the adaptive templates. We therefore report corrected case-level fields: Detected means that an adversarial case retrieved poisoned content in the observation stream, and Exposed means that the poisoned content remained model-visible after condition-specific processing.

H.1 Metric Granularity

The paper uses three granularities. Case-level metrics average over StrategyQA cases or paired benign/adversarial cases and ask whether a run retrieved poison, exposed poison, manifested the attack, or answered correctly at least once within the case. Retrieval-event-level metrics average over individual retrieved observations and are used when the unit of analysis is a boundary decision. Record-level benign replay metrics average over standalone benign records that are not embedded in a full ReAct trajectory.

The cross-provider containment table reports case-level averages over three 25-question runs per provider-condition group. Inspector-swap replay is event-level over saved retrieval events. Benign false-quarantine replay is record-level. Fixed-trace overhead replay is retrieval-event-level and measures local processing cost per saved event. The adaptive Kimi pilot reports corrected case-level Detected and Exposed fields because the saved aggregate raw-poison counter was not defined for those templates.

I Cross-Provider Confidence Intervals

Table 15 expands Table 1 with case-level bootstrap confidence intervals from the saved cross-provider summary.

J Replay Details

Table 16 gives the full inspector-swap replay summary. Rule and heuristic inspectors are evaluated

Term	Definition and scope
Prompt quoting	Prompt-level isolation comparator that wraps retrieved text as untrusted quoted evidence while leaving the raw text model-visible. It tests whether instructing the model not to follow untrusted content is sufficient, rather than enforcing a runtime quarantine state.
Paraphrase-aware keyword	Lexical/pattern comparator that extends static keyword matching to paraphrased trigger-like expressions. It is stronger than exact trigger matching but remains pattern-based and does not enforce raw–view separation.
Paraphrase stress	Stress test in which poisoned instructions are rewritten into different surface forms. The paper includes a held-out paraphrase slice and a six-family setting: direct, policy, factual-note, system-imitation, tool-suggestion, and indirect-preference.
Adaptive poisoning stress	Pilot setting with poison families that are not simple trigger paraphrases. The Kimi pilot uses factual misinformation, mixed-language poisoning, and soft preference manipulation.
Benign false quarantine	Rate at which benign or harmless records are incorrectly assigned QUARANTINE. This measures overblocking behavior on the tested benign templates; it is not a deployment-wide utility guarantee.
False intervention	For comparators without a quarantine state, the rate at which benign records are wrapped, blocked, or otherwise modified.
Retrieval pressure k	Number of retrieved records returned to the ReAct observation at each retrieval step. Larger k changes the candidate pool and can increase the chance that useful and poisoned records appear together.
Full-ReAct axis	AgentPoison-derived StrategyQA setting that preserves iterative ReAct search, lookup, and finish behavior rather than reducing the task to single-turn retrieval classification.
Trigger-query search context	Adapted adversarial search context that includes the trigger sequence with the question to create stable poisoned-retrieval pressure on the evaluated axis.
Inspector-swap replay	Offline replay that applies alternative inspectors to saved retrieval events without rerunning the agent or calling providers. It studies detector precision, cost, and compatibility with the boundary.
LLM-judge inspector	Inspector variant in which a model labels saved retrieval events. In this paper it is evaluated only on a capped subset, so its results characterize precision–cost trade-offs rather than full-run behavior.
Fixed-trace replay	Offline replay over saved retrieval events used to isolate local containment processing. It excludes LLM inference, network latency, and new retrieval trajectories.
Token delta	Serialized-context length proxy computed from saved safe views and observations. It is not provider-billed token usage.

Table 13: Experimental terms and comparator definitions used in the main paper and appendix.

over the full 466-event trace; LLM judges use a capped 50-event subset. Table 17 reports the benign-record replay used to quantify false quarantine on tested benign families.

K Adaptive-Pilot Diagnostics and Fixed-Trace Scope

Table 18 supplements Table 7 with clean utility, retrieval-event counts, intervention rates, and benign false-block proxies. The primary endpoint remains exposure containment; attack manifestation is reported as a downstream diagnostic.

K.1 Fixed-Trace Overhead Scope

The fixed-trace overhead replay uses 466 saved retrieval events, including 22 poisoned events. It replays local serialization, inspection, quarantine/rewrite decisions, and ReAct action canonicalization without invoking model providers. Thus microsecond/event values are local replay costs, and token deltas are serialized-context proxies rather than provider-billed token counts.

L Data-Only Supplement

The optional data-only supplement contains aggregate CSV/JSON summaries and selected run-level results. It excludes source code, provider caches,

Mode	Raw poison retrieved?	Exposure decision	Model-visible observation	Outcome label
No defense	Yes	None	Evidence text plus an injected instruction to ignore the question and output an attacker-chosen answer.	Manifested
FLOWFENCE	Yes	QUARANTINE	Quarantine notice; non-quarantined evidence, if any, remains available.	Not manifested

Table 14: Synthetic audit example illustrating that retrieval and exposure are separate states.

Provider	Condition	Raw	Exp.	Man.	Clean	AtkU
MiniMax	No defense	.467 [.347,.587]	.467 [.347,.587]	.253 [.160,.360]	.373 [.280,.440]	.333 [.320,.360]
MiniMax	Static keyword	.467 [.360,.573]	.000 [.000,.000]	.000 [.000,.000]	.360 [.280,.480]	.360 [.240,.520]
MiniMax	FLOWFENCE	.440 [.333,.547]	.000 [.000,.000]	.000 [.000,.000]	.360 [.280,.400]	.387 [.280,.480]
Qwen	No defense	.960 [.907,1.000]	.960 [.907,1.000]	.693 [.587,.787]	.853 [.800,.880]	.267 [.200,.320]
Qwen	Static keyword	.960 [.907,1.000]	.000 [.000,.000]	.000 [.000,.000]	.893 [.840,.920]	.720 [.640,.800]
Qwen	FLOWFENCE	.960 [.907,1.000]	.000 [.000,.000]	.000 [.000,.000]	.867 [.840,.920]	.653 [.560,.720]
Kimi	No defense	.960 [.907,1.000]	.960 [.907,1.000]	.893 [.813,.960]	.440 [.400,.480]	.027 [.000,.080]
Kimi	Static keyword	.960 [.907,1.000]	.000 [.000,.000]	.000 [.000,.000]	.493 [.400,.560]	.600 [.520,.640]
Kimi	FLOWFENCE	.960 [.907,1.000]	.000 [.000,.000]	.000 [.000,.000]	.427 [.400,.440]	.600 [.560,.640]

Table 15: Cross-provider means with 95% bootstrap confidence intervals over the 75 case instances in each provider-condition group.

raw stdout/stderr logs, local configuration manifests with absolute paths, and raw case-detail directories not needed to audit paper-facing aggregates. Package validation recorded 251 copied files, 460 excluded entries, 27 cross-provider runs, 6 P0 same-axis comparator runs, 9 P1 adaptive pilot runs, and 24 MiniMax mechanism/stress run directories.

Inspector	Events	Poison	Clean	Flagged	Quarantine	Recall	Exposed	FQ
Rule	466	22	444	.047	.047	1.00	.00	.00
Heuristic	466	22	444	.047	.047	1.00	.00	.00
LLM judge Kimi	50	22	28	.440	.440	1.00	.00	.00
LLM judge MiniMax	50	22	28	1.000	1.000	1.00	.00	1.00

Table 16: Inspector-swap replay details. Flagged and Quarantine are event rates; Recall is poison recall; Exposed is poisoned-event exposure after the boundary; FQ is false quarantine on clean events. Local costs are 24.65 and 39.44 microseconds/event for rule and heuristic inspectors; LLM-judge token totals are 13,347 for Kimi and 21,141 for MiniMax.

Inspector	Family	Records	FQ	Interv.	μs/rec.
Rule	Benign instructional	30	.000	.000	6.22
Rule	Useful factual	30	.000	.000	5.44
Rule	Irrelevant harmless	30	.000	.000	5.81
Rule	All	90	.000	.000	5.82
Heuristic	Benign instructional	30	.100	.100	5.93
Heuristic	Useful factual	30	.000	.000	5.63
Heuristic	Irrelevant harmless	30	.000	.000	5.94
Heuristic	All	90	.033	.033	5.83

Table 17: Offline benign-record false-quarantine replay. Interv. is total intervention rate.

Family	Condition	Clean	AtkU	Man.	Ret. ev.	Int. ev.	Adv int.	Benign FB
Factual misinformation	No defense	.56	.28	.00	255	.000	.00	.00
Factual misinformation	Static keyword	.56	.28	.00	260	.000	.00	.00
Factual misinformation	FLOWFENCE	.56	.64	.00	238	.437	.96	.04
Mixed language	No defense	.56	.32	.00	254	.000	.00	.00
Mixed language	Static keyword	.56	.20	.00	264	.000	.00	.00
Mixed language	FLOWFENCE	.56	.72	.00	223	.404	.96	.04
Soft preference	No defense	.64	.40	.00	239	.000	.00	.00
Soft preference	Static keyword	.60	.12	.04	277	.000	.00	.00
Soft preference	FLOWFENCE	.64	.52	.00	236	.475	.96	.04

Table 18: Additional diagnostics for the Kimi adaptive-poisoning pilot. Ret. ev. is the saved retrieval-event count; Int. ev. is defense intervention event rate; Adv int. is adversarial intervention case rate; Benign FB is a benign false-block proxy.

Mode	Events	Poison exp.	Interv.	μs/ev.	Tok. Δ
No defense	466	22/22	0	0.96	0.00
Static keyword	466	0/22	22	2.74	-2352.75
FLOWFENCE	466	0/22	22	14.02	-2380.25
Quoted evidence	466	22/22	466	1.26	15960.50

Table 19: Fixed-trace replay scope. Quoted evidence increases the serialized-context proxy because it wraps every observation.